

E-STORE

Sécurité JWT & MongoDB

Authentification, autorisation et persistance documentaire

Module	Full-Stack
Encadrant	Pr. Omar Zahour
Établissement	FSBM — Université Hassan II
Année	2025-2026
Auteurs	Akram Belmoussa & Nouhaila Ben Soumane
Document	3/7 d'une série explicative

1. Vue d'ensemble de la sécurité

L'application utilise **Spring Security 6** en mode **stateless** (pas de session), avec authentification par **JWT** (JSON Web Token) signés en HS256. Les mots de passe sont hashés avec **BCrypt**.

Endpoints publics vs protégés

Endpoint	Accès	Détail
POST /api/auth/register	Public	Inscription
POST /api/auth/login	Public	Renvoie le JWT
GET /api/products/**	Public	Catalogue lisible sans auth
GET /api/categories/**	Public	Idem
GET /api/reviews/**	Public	Avis consultables sans auth
POST /api/products	ADMIN	@PreAuthorize("hasRole('ADMIN')")
GET /api/cart	Authentifié	Panier de l'utilisateur courant
POST /api/orders	Authentifié	Validation de la commande

2. JwtService — génération du token

```
public String generateToken(User user) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("uid", user.getId());
    claims.put("role", user.getRole().name());
    claims.put("name", user.getFirstName() + " " + user.getLastName());

    Date now = new Date();
    Date exp = new Date(now.getTime() + expirationMs); // 24h

    return Jwts.builder()
        .claims(claims).subject(user.getEmail())
        .issuedAt(now).expiration(exp)
        .signWith(key).compact(); // signature HMAC-SHA256
}
```

Le token contient l'email (subject), l'id utilisateur, son rôle et son nom complet. Il est signé avec une clé secrète (configurée dans application.properties) et expire au bout de **24 heures**.

3. JwtAuthenticationFilter — filtre Spring

```
@Override
protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                FilterChain chain) {
    String header = req.getHeader("Authorization");
    if (header == null || !header.startsWith("Bearer ")) {
        chain.doFilter(req, res); return;
    }
    String token = header.substring(7);
    String email = jwtService.extractEmail(token);
    if (email != null && SecurityContextHolder.getContext().getAuthentication() == null) {
        UserDetails user = userDetailsService.loadUserByUsername(email);
        if (jwtService.isTokenValid(token, email)) {
            UsernamePasswordAuthenticationToken auth =
```

```
        new UsernamePasswordAuthenticationToken(user, null, user.getAuthorities());
        SecurityContextHolder.getContext().setAuthentication(auth);
    }
}
chain.doFilter(req, res);
}
```

4. Côté frontend — AuthInterceptor

Toutes les requêtes HTTP passent par l'intercepteur, qui ajoute automatiquement le token au header Authorization. Aucun composant n'a à le gérer manuellement.

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {  
  const auth = inject(AuthService);  
  const token = auth.token;  
  if (!token) return next(req);  
  return next(req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })));  
};
```

5. AuthGuard — protection des routes

```
export const authGuard: CanActivateFn = () => {  
  const auth = inject(AuthService);  
  const router = inject(Router);  
  if (auth.isAuthenticated) return true;  
  router.navigate(['/login']);  
  return false;  
};
```

Appliqué aux routes **/cart**, **/orders** et **/profile** dans app.routes.ts via canActivate: [authGuard].

6. MongoDB — domaine review

Les avis utilisateurs sont stockés dans **MongoDB** plutôt qu'en SQL pour illustrer la coexistence de deux paradigmes de persistance. Cela répond à un cas d'usage typique : données semi-structurées, volumineuses, lues fréquemment mais peu contraintes relationnellement.

Document Mongo

```
@Document(collection = "reviews")  
public class Review {  
  @Id private String id;  
  @Indexed private Long productId;  
  @Indexed private Long userId;  
  private String authorName;  
  private int rating; // 1 à 5  
  private String comment;  
  private Instant createdAt;  
}
```

Repository réactif Mongo

```
public interface ReviewRepository extends MongoRepository<Review, String> {  
  List<Review> findByProductIdOrderByCreatedAtDesc(Long productId);  
  List<Review> findByUserIdOrderByCreatedAtDesc(Long userId);  
}
```

Spring Data MongoDB génère automatiquement la requête à partir du nom de la méthode — aucune SQL ni JPQL à écrire.

Exemple de document JSON stocké

```
{  
  "_id": ObjectId("671f3a2..."),  
  "productId": 1,  
  "userId": 2,  
  "authorName": "Nouhaila Test",  
  "rating": 5,  
}
```

```
"comment": "Excellent ordinateur, performances au top !",  
"createdAt": ISODate("2026-04-28T14:32:11.123Z")  
}
```